SYRIAN VIRTUAL UNIVERSITY

Intelligent Algorithms | F25

PROJECT REPORT

Exam Scheduling Optimization

No.	Student Name	Student ID	Year of Study
1	Hala Nejmeh	hala_274110	C5
2	Nour sail alhalaqi	nour_252300	C7
3	Sarah Abdulrazzak	sarah_334080	C4
4	Sara samer algeroude	sara_264265	C7
5	Ayat AlSheikh	ayat_238172	C5
6	weaam omar jeha	weaam_242969	C3
7	Esraa Said Alshurbaji	esraa_246565	C3

1. Introduction and Problem Description

University exam scheduling is an optimization problem that requires assigning each exam to an available timeslot and room. Creating a suitable schedule can be difficult because students may have several exams, while the number of rooms and available timeslots is limited.

The main objectives of this project are:

- **Satisfy Hard Constraints:** Generate valid schedules without student conflicts, room conflicts, or room capacity violations.
- **Minimize Soft Penalties:** Reduce multiple exams on the same day, consecutive exams, and uneven exam distribution.
- **Compare Optimization Algorithms:** Evaluate Genetic Algorithm (GA), Ant Colony Optimization (ACO), and Particle Swarm Optimization (PSO) based on solution quality and performance.
- **Use Dynamic Programming:** Test DP on the Small dataset and compare its result with the other algorithms.

2. Problem Formulation

2.1 Solution Representation

In this project, an exam schedule is represented using a Python dictionary. Each exam is assigned to a specific timeslot and room.

The schedule contains three main elements:

- **Exam ID:** Identifies the exam.
- **Timeslot ID:** Specifies when the exam takes place.
- **Room ID:** Specifies where the exam takes place.

The representation follows this format:

exam_id: (timeslot_id, room_id)

For example: {"E005": ("T001", "R01")}

This means that exam E005 is scheduled in timeslot T001 and room R01.

Each exam must be assigned exactly once, using a valid timeslot and room. The same solution representation is used by all optimization algorithms to ensure a consistent comparison.

2.2 Hard Constraints

Hard constraints are mandatory rules that every exam schedule must satisfy. A schedule is considered invalid if any of these constraints is violated.

The project includes five hard constraints:

- **H1 – Student Conflict:** A student cannot have two exams scheduled in the same timeslot.
- **H2 – Room Conflict:** Two exams cannot use the same room during the same timeslot.
- **H3 – Room Capacity:** The assigned room must have enough capacity for all students registered in the exam.
- **H4 – Complete Assignment:** Every exam must be assigned exactly once.
- **H5 – Valid Assignments:** Each exam must be assigned to an existing timeslot and room.

All generated schedules are checked using the validate_schedule() function from main.py to ensure that these constraints are satisfied.

2.3 Soft Constraints

Soft constraints are preferences that help improve the quality of an exam schedule. Unlike hard constraints, violating them does not make the schedule invalid, but it increases the fitness value.

The project includes four soft constraints:

Penalty	Description
P1 – Multiple Exams on the Same Day	Counts each student-day where a student has more than one exam. Each student-day is penalized only once.
P2 – Consecutive Exam Periods	Counts pairs of a student's exams scheduled in consecutive periods on the same day.
P3 – Uneven Exam Distribution	Measures the uneven distribution of exams across days using the sum of squared differences from the average daily exam count.
P4 – Exams on Consecutive Days	Counts pairs of a student's exams scheduled on two consecutive days.

The objective is to minimize these penalties while satisfying all hard constraints.

2.4 Objective Function

The objective function evaluates the quality of a valid exam schedule by calculating the weighted sum of the four soft penalties.

The fitness function is defined as:

Fitness = 3 × P1 + P2 + P3 + 4 × P4

The weights assigned to each penalty are:

Penalty	Weight
P1 – Multiple Exams on the Same Day	3
P2 – Consecutive Exam Periods	1
P3 – Uneven Exam Distribution	1
P4 – Exams on Consecutive Days	4

Higher weights were assigned to P1 and P4 to give more importance to reducing student workload and exams on consecutive days. These weights were selected based on our design priorities, rather than through a separate weight optimization experiment.

The goal is to minimize the fitness value while satisfying all hard constraints. A lower fitness value indicates a better schedule according to the defined objective function.

The same fitness function is used by all four algorithms to ensure a consistent comparison.

3. Optimization Algorithms

3.1 Genetic Algorithm (GA)

The Genetic Algorithm (GA) is an optimization algorithm inspired by natural selection. It works with a population of exam schedules and improves their quality over several generations.

Our GA implementation follows these main steps:

1. **Initial Population:** Generate valid exam schedules using randomized backtracking.
2. **Selection:** Select parents using Tournament Selection based on their fitness values.
3. **Crossover:** Combine two parents using One-Point Crossover to produce a new schedule.
4. **Repair:** Fix invalid assignments by restoring assignments from the first parent when necessary.
5. **Mutation:** Attempt to swap exam assignments while maintaining the hard constraints.
6. **Fitness Evaluation:** Calculate the weighted soft penalties and track the best solution found.

Hyperparameters: Population size, number of generations, tournament size, crossover rate, and mutation rate.

Stopping Criterion: The algorithm stops after completing the specified number of generations.

The algorithm was tested on Small, Medium, and Large datasets. Different parameter settings were evaluated to select suitable configurations for each dataset.

3.2 Ant Colony Optimization (ACO)

Ant Colony Optimization (ACO) is an optimization algorithm inspired by the way ants use pheromone trails to find good paths. It builds exam schedules using pheromone values and heuristic information to guide the selection of assignments.

Our ACO implementation follows these main steps:

1. **Pheromone Initialization:** Initialize pheromone values for possible exam assignments.
2. **Heuristic Information:** Estimate the quality of possible assignments to guide the ants.
3. **Solution Construction:** Each ant builds an exam schedule by selecting feasible assignments while respecting hard constraints.
4. **Fitness Evaluation:** Calculate the weighted soft penalties and track the best schedule found.
5. **Pheromone Evaporation:** Reduce pheromone values to limit the influence of previous choices.
6. **Pheromone Update:** Reinforce pheromone values based on the quality of the solutions.

Hyperparameters: Number of ants, number of iterations, alpha (pheromone influence), beta (heuristic influence), evaporation rate, and q (pheromone reinforcement factor).

Stopping Criterion: The algorithm stops after completing the specified number of iterations.

The algorithm was tested on Small, Medium, and Large datasets. Different parameter settings were evaluated to select suitable configurations for each dataset.

3.3 Particle Swarm Optimization (PSO)

Particle Swarm Optimization (PSO) is an optimization algorithm that uses a group of particles to search for better solutions. In our project, each particle represents an exam schedule.

Our PSO implementation follows these main steps:

1. **Swarm Initialization: Generate valid exam schedules and initialize the particles.**
2. **Global Best Initialization: Find the best schedule among the initial particles.**
3. **Velocity Update: Create assignment changes using the previous velocity, personal best, and global best.**
4. **Position Update: Apply valid changes to the schedules. If a change is invalid, the algorithm tries a swap.**

5. **Personal Best Update: Update each particle's best schedule when a lower fitness is found.**
6. **Global Best Update: Keep track of the best schedule found by the swarm.**

Hyperparameters: Number of particles, number of iterations, inertia weight (w), cognitive coefficient (c1), and social coefficient (c2).

Stopping Criterion: The algorithm stops after completing the specified number of iterations.

The algorithm was tested on Small, Medium, and Large datasets. Different parameter settings were tested to select suitable values for each dataset.

3.4 Dynamic Programming (DP)

Dynamic Programming (DP) is an optimization approach that divides a problem into smaller subproblems and stores their results to avoid repeated calculations.

We implemented DP on the Small dataset. The algorithm builds the exam schedule day by day and uses memoization to avoid repeating calculations.

Our DP implementation follows these main steps:

1. **Daily Assignment Generation:** Generate possible exam assignments for each day while avoiding student conflicts.
2. **State Representation:** Represent each state using the current day, previously assigned exams, and exams scheduled on the previous day.
3. **Fitness Calculation:** Calculate the daily contributions to P1, P2, P3, and P4 using the same objective function as the other algorithms.
4. **Recurrence and Memoization:** Explore valid daily assignments, select the minimum total cost, and store calculated states using memoization.
5. **Schedule Reconstruction:** Reconstruct the best schedule and assign available rooms with sufficient capacity.
6. **Final Validation:** Check all hard constraints and verify the final fitness using the shared functions in main.py.

Stopping Criterion: The algorithm finishes when all days have been processed and all exams have been assigned. Incomplete schedules are rejected.

Hyperparameters: This implementation does not use random seeds or tunable optimization hyperparameters.

DP was implemented and tested on the Small dataset only. Medium and Large were not executed because extending the current approach would require exploring a much larger number of possible states and daily assignments.

4. Experimental Setup

4.1 Datasets

Three dataset sizes were used to evaluate the algorithms.

Dataset	Small	Medium	Large
Exams	10	40	100
Students	30	100	300
Rooms	3	6	12

Dataset	Small	Medium	Large
Days	5	10	20
Periods per Day	2	3	4
Timeslots	10	30	80

These datasets were used to study how problem size affects solution quality and execution time.

4.2 Experimental Procedure

We used the Small, Medium, and Large datasets to evaluate GA, ACO, and PSO.

Different parameter settings were tested to select suitable configurations for each algorithm. The selected configurations were evaluated using three random seeds: 42, 43, and 44.

For each experiment, we recorded the fitness value, execution time, and hard constraint violations.

Dynamic Programming (DP) was tested on the Small dataset only. It does not use random seeds or parameter tuning.

Selected Hyperparameters

We used the following parameter settings for the final experiments.

Algorithm	Parameter	Small	Medium	Large
GA	Population Size	40	10	40
GA	Generations	50	50	50
GA	Tournament Size (k)	3	2	3
GA	Crossover Rate	0.6	1.0	1.0
GA	Mutation Rate	0.5	1.0	0.3
ACO	Number of Ants	5	20	5
ACO	Iterations	10	10	10
ACO	Alpha	1	1	1
ACO	Beta	3	3	3
ACO	Evaporation Rate	0.2	0.2	0.2
ACO	Q	2	2	2
PSO	Number of Particles	10	20	20
PSO	Iterations	20	20	20
PSO	Inertia Weight (w)	0.5	0.8	0.8
PSO	Cognitive Coefficient (c1)	1	2	2
PSO	Social Coefficient (c2)	2	1	1

4.3 Evaluation Metrics

We used the following metrics to evaluate and compare the algorithms:

Metric	Description
Solution Quality (Fitness)	The weighted soft penalty of the final schedule. Lower values indicate better solutions.
Execution Time	The time required to run the algorithm, measured in seconds.
Hard Constraint Violations	The number of hard constraint violations in the final schedule. A valid schedule must have zero violations.
Convergence	How the best fitness changes over generations or iterations.
Stability	How consistent the final fitness values are across repeated runs with different random seeds.

For GA, ACO, and PSO, convergence and stability are evaluated using the recorded experimental results.

For DP, we report the final fitness, execution time, and memoization statistics. Convergence and random seed stability are not applicable to our deterministic DP implementation.

5. Experimental Results

5.1 Genetic Algorithm (GA)

The following table shows the average final fitness and execution time for GA on the three datasets using the selected parameters.

Dataset	Average Final Fitness	Average Execution Time (s)
Small	272.67	1.0723
Medium	395.33	1.5377
Large	587.00	31.0940

The results show that execution time increased as the dataset size increased.

For the Small dataset, the final fitness values ranged from 270 to 276, with a sample standard deviation of 3.06. All three runs produced valid schedules.

The convergence results showed that GA improved the best fitness during the generations. The detailed convergence curves will be presented in the comparison section.

5.2 Ant Colony Optimization (ACO)

The following table shows the average final fitness and execution time for ACO on the three datasets using the selected parameters.

Dataset	Average Final Fitness	Average Execution Time (s)
Small	274.67	0.0208
Medium	345.33	4.0810
Large	431.33	39.4300

The results show that execution time increased as the dataset size increased, especially for the Large dataset.

All recorded final schedules satisfied the hard constraints, with zero violations.

The fitness values and convergence histories were recorded during the experiments to evaluate solution quality and algorithm performance.

5.3 Particle Swarm Optimization (PSO) Results

The following table shows the average final fitness and execution time for PSO on the three datasets using the selected parameters.

Dataset	Average Final Fitness	Average Execution Time (s)
Small	279.33	0.0325
Medium	413.00	1.4919
Large	710.00	4.3084

The execution time increased as the dataset size increased.

For the Small dataset, the final fitness values ranged from 274 to 286, with a sample standard deviation of 6.11.

All recorded final schedules satisfied the hard constraints, with zero violations.

The convergence histories showed that the best fitness stayed the same or decreased during the iterations.

5.4 Dynamic Programming (DP) Results

Dynamic Programming was tested on the Small dataset using memoization.

The results are summarized below:

Metric	Result
Final Fitness	258.00
Execution Time (s)	8.0668
Hard Constraint Violations	0
DP States	51,393
Cache Hits	976,330

The algorithm found a valid schedule for all 10 exams, with zero hard constraint violations.

DP achieved a fitness of 258.00 on the Small dataset. We used this result to compare DP with the other algorithms.

Since DP is deterministic and does not use iterative generations, no convergence curve or random-seed stability analysis was required.

6. Algorithms Comparison

6.1 Fitness Comparison

We compared the final fitness values of the algorithms on the three datasets. Lower fitness values indicate better schedules.

Dataset	GA	ACO	PSO	DP
Small	272.67	274.67	279.33	258.00
Medium	395.33	345.33	413.00	—
Large	587.00	431.33	710.00	—

For the Small dataset, DP achieved the lowest fitness. Among GA, ACO, and PSO, GA had the lowest average fitness.

For Medium and Large, ACO achieved the lowest average fitness.

DP was tested on the Small dataset only.

6.2 Execution Time Comparison

We compared the average execution time of the algorithms on the three datasets. Lower execution time means the algorithm runs faster.

Dataset	GA (s)	ACO (s)	PSO (s)	DP (s)
Small	1.0723	0.0208	0.0325	8.0668
Medium	1.5377	4.0810	1.4919	—
Large	31.0940	39.4300	4.3084	—

ACO had the shortest average execution time on the Small dataset.

For Medium, PSO and GA had similar execution times, with PSO being slightly faster.

For Large, PSO was the fastest among the three algorithms.

DP was tested on the Small dataset only.

6.3 Convergence Comparison

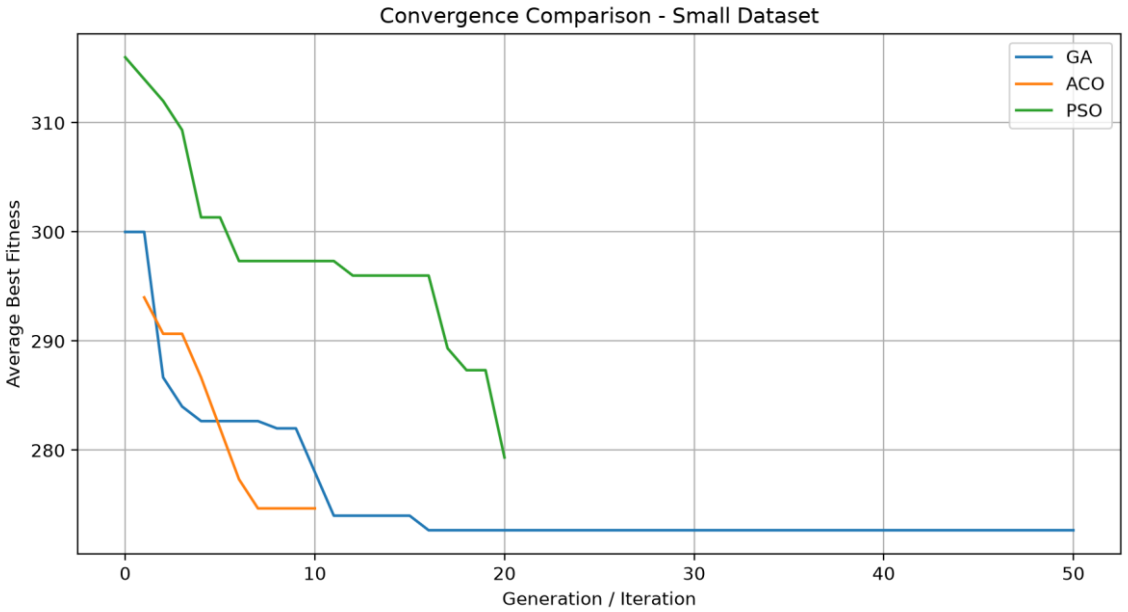
We used the recorded convergence histories to see how the best fitness changed during the execution of each algorithm.

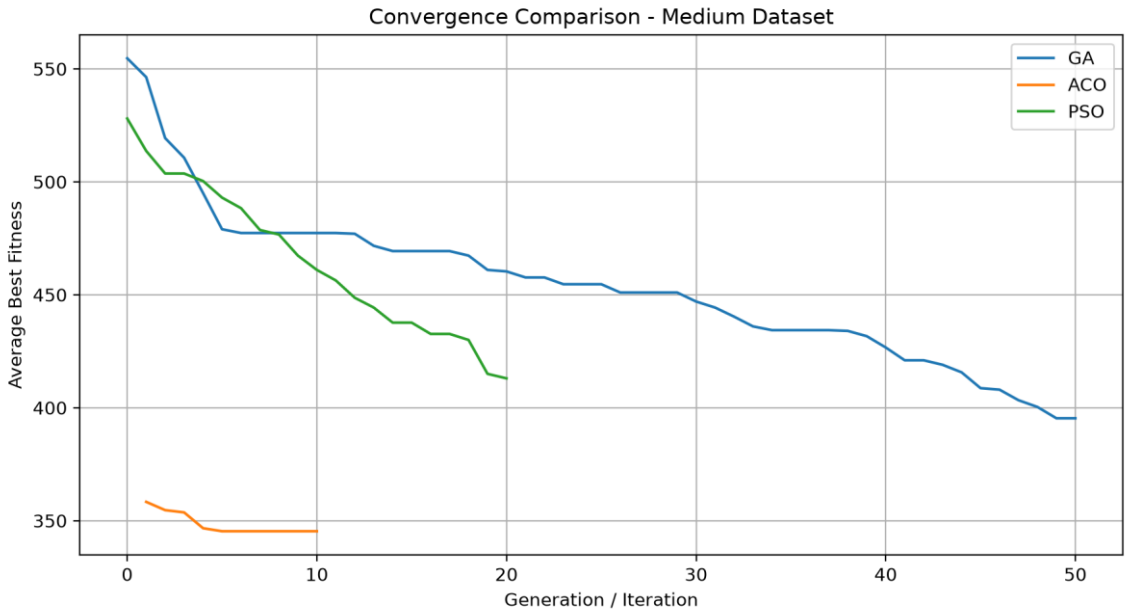
GA was tested using 50 generations, PSO using 20 iterations, and ACO using 10 iterations.

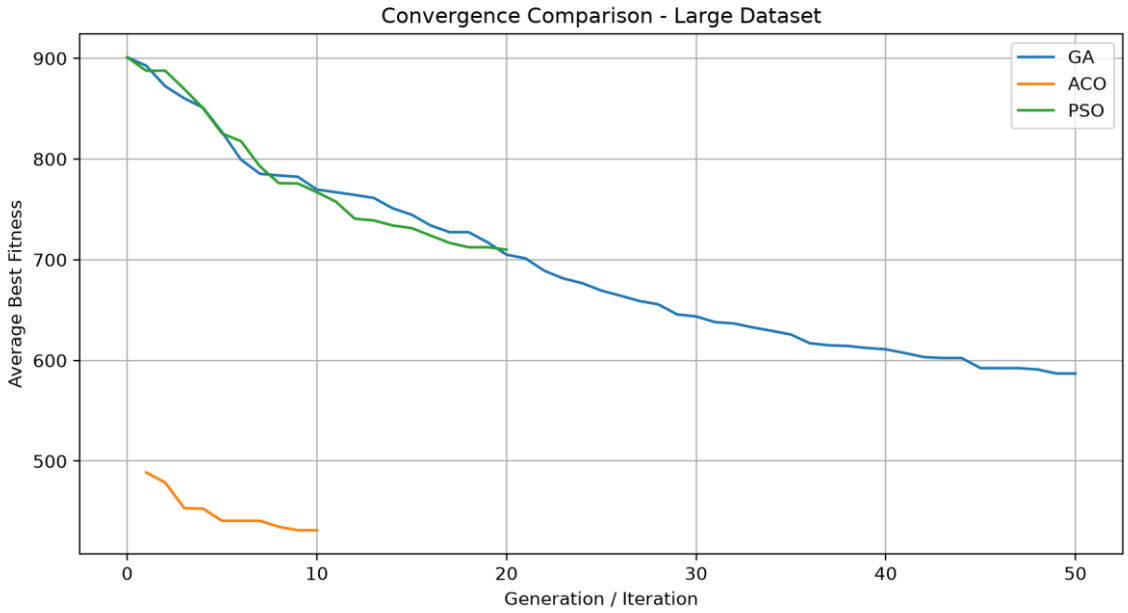
The results show that the best fitness decreased or stayed the same during execution.

GA and PSO improved their solutions over several steps.

The algorithms use different numbers of generations or iterations, so we also considered execution time when comparing their performance.







6.4 Stability Comparison

We used the sample standard deviation of the final fitness values to compare the stability of the algorithms.

A lower standard deviation means less variation between runs.

Dataset	GA	ACO	PSO
Small	3.06	7.57	6.11
Medium	28.92	1.53	8.19
Large	11.53	23.76	20.81

GA had the lowest variation on Small and Large.

ACO had the lowest variation on Medium.

The results show that stability changed depending on the algorithm and dataset size.

Since we used only three runs for each configuration, these results give a limited view of stability.

7. Discussion and Conclusion

7.1 Discussion

The experiments showed that the algorithms performed differently on each dataset.

GA achieved the lowest average fitness among GA, ACO, and PSO on the Small dataset. ACO achieved better average fitness on Medium and Large.

ACO produced better schedules on the larger datasets, but it required more execution time than PSO. PSO was faster on Large, but its average fitness was higher.

The stability results also varied. GA had the lowest variation on Small and Large, while ACO had the lowest variation on Medium.

These differences may be related to how each algorithm searches for solutions and the parameters used during the experiments.

DP achieved the lowest recorded fitness on Small, but we did not test it on Medium or Large because of the expected computational complexity.

All final schedules satisfied the hard constraints.

The results show that no single algorithm was the best in every aspect. We need to consider solution quality, execution time, and stability when comparing them.

7.2 Conclusion

This project helped us test different ways of solving the exam scheduling problem.

By comparing the results, we learned how the dataset size and parameter settings can affect the performance of each algorithm.

8. Team Contributions

All team members contributed to proposing project ideas, project integration, and testing.

The following table summarizes the individual contributions of each team member.

Student Name	Contribution
Hala Nejmeh	Genetic Algorithm (GA) implementation, experiments and parameter tuning, GA documentation, README contribution, report writing, results visualization, and algorithms comparison.
Esraa Said Alshurbaji	Particle Swarm Optimization (PSO) implementation and experiments.
Weaam Omar Jeha, Sara Samer Algeroude, Ayat AlSheikh	All three members worked together on ACO implementation, experiments, parameter tuning, documentation, and results analysis.
Nour Sail Alhalaqi	Dynamic Programming (DP) implementation and experiments.
Sarah Abdulrazzak	Dataset preparation, schedule generation and validation, constraints, and penalty calculations in main.py.